

Rune Transformation Power

Problem Statement: A sorcerer has discovered a magical sequence of runes, represented as a string made up of two types of runes: X and Y. The sorcerer wants to maximize their magical power by transforming the sequence into more potent forms. They can perform the following transformations:

1. Pick a substring "XY", transform it into "YZ", and gain one unit of magical power.
2. Pick a substring "YX", transform it into "ZY", and gain one unit of magical power.

Your task is to find out the maximum magical power the sorcerer can accumulate by performing these transformations optimally. Constraints:

$1 \leq |s| \leq 200000$

The string contains only characters X and Y

Input Format:

A single line containing the string s.

Output Format:

Print a single integer representing the maximum magical power obtainable.

Sample Input:

YXYYXY

Sample Output:

2

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; cin >> s;
    int cX = 0, cSX = 0, cSY = 0, smallSeg = 1e9;
    bool flg = false;
    for(int i = 0; i < s.size(); i++) {
        if(s[i] == 'X') {
            int cnt = 0;
            while(i < s.size() && s[i] == 'X') { cX++; cnt++; i++; }
            i--;
            cSX++;
            smallSeg = min(smallSeg, cnt);
        } else {
            int cnt = 0;
            while(i < s.size() && s[i] == 'Y') { cnt++; i++; }
            i--;
            if(cnt > 1) flg = true;
            cSY++;
        }
    }
    if(cSX <= cSY) cout << cX << endl;
    else if(flg) cout << cX << endl;
    else cout << cX - smallSeg << endl;
    return 0; }
```

Maximum Weight Node

Problem Statement:

The task is to find the cell with maximum weight.

The weight of a cell is defined as the sum of the indices of all cells pointing to that cell.

If there are multiple cells with the same maximum weight, return the cell with the highest index.

Note:

- Cells are indexed from 0 to N-1.
- If no cell points to the *i*th cell, then its weight is 0.

Input Format:

- First line contains an integer N.
- Second line contains N integers representing the array `edge[]`.
- `edge[i]` contains the cell number that can be reached from cell *i* in one step.
- `edge[i] = -1` means the *i*th cell has no outgoing edge.

Output Format:

Print the node number having maximum weight.

Sample Input:

```
23
4 4 1 4 13 8 8 8 0 8 14 9 15 11 -1 10 15 22 22 22 22 22 21
```

Sample Output:

```
22
```

Sample Input:

```
7
4 4 -1 2 2 3 4
```

Sample Output:

```
4
```

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> child(n), weight(n, 0);
    for(int i = 0; i < n; i++) {
        cin >> child[i];
        if(child[i] != -1) weight[child[i]] += i; }
    int ans = -1, mxWeight = -1;
    for(int i = 0; i < n; i++) {
        if(weight[i] >= mxWeight) {
            mxWeight = weight[i];
            ans = i; } }
    cout << ans << endl;
    return 0; }
```

Gridopolis Critical Blocks

Problem Statement:

In the city of Gridopolis, the city is arranged in a block formation with 2 parallel streets and n intersections (blocks) along each street.

Some blocks are open and accessible (represented by '.') while others are closed for construction (represented by 'x').

A block is accessible if it has a free passage on at least one of the streets.

Residents of Gridopolis want to ensure that their neighborhoods remain connected.

Two blocks are part of the same neighborhood if:

- they are accessible, and
- they can be reached through adjacent blocks either horizontally or vertically.

Your task is to identify critical blocks in the city grid:

- a block is critical if closing it would divide the city into exactly 3 separate neighborhoods.

Input Format:

- First line contains an integer n ($1 \leq n \leq 200000$)
- Next two lines contain strings of length n
- Each character is either:
 - '.' → open block
 - 'x' → blocked cell

Output Format:

Print a single integer representing the number of blocks that, if closed, would split the city into exactly 3 connected neighborhoods.

Additional Constraint:

Initially, there is at most one connected neighborhood in the grid.

Sample Input:

```
8
.....X..
.x.xx...
```

Sample Output:

```
1
```

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    string s1, s2;
    cin >> s1 >> s2;
    int ans = 0;
    for(int i = 1; i < n - 1; i++) {
        if(s1[i] == '.' && s1[i - 1] == '.' && s1[i + 1] == '.' && s2[i] == '.' && s2[i - 1] == 'x' &&
s2[i + 1] == 'x') ans++;
        if(s2[i] == '.' && s2[i - 1] == '.' && s2[i + 1] == '.' && s1[i] == '.' && s1[i - 1] == 'x' &&
s1[i + 1] == 'x') ans++;
    }
    cout << ans << endl;
    return 0; }
```

The Kingdom's Rebellion

Problem Statement:

In the Kingdom of Arborvale, there is a hierarchical system of nobles arranged in a tree-like structure. The kingdom has n nobles, each assigned a unique number from 1 to n .

At the top of the hierarchy is the King, who is considered the root of the kingdom's structure.

Each noble has a parent, except the King, who has no parent.

The nobles have ancestral respect rules:

- **Respects:** A noble respects its ancestors if it is loyal to the King's rule.
- **Rebels:** Some nobles, however, do not respect their ancestors and openly defy them.

Given the tension in the kingdom, you decide to remove rebellious nobles one by one to restore peace.

Removal Process:

1. Select a non-root noble that is rebellious (does not respect its parent) and has no children that respect it.
2. If there are multiple such nobles, select the one with the smallest index.
3. When a rebellious noble is removed, all its children are immediately reassigned to its parent.
4. Continue until no more nobles satisfy the removal condition.

Your task is to determine the order in which rebellious nobles will be removed.

Input Format:

- First line contains integer n ($1 \leq n \leq 100$)
- Next n lines contain two integers:
 - $p_i \rightarrow$ parent of node i
 - $r_i \rightarrow$ whether node i respects parent

Where:

- $p_i = -1$ means node i is the root
- $r_i = 0$ means node respects parent
- $r_i = 1$ means node does not respect parent

It is guaranteed that the given parents form a valid tree.

Output Format:

- Print indices of nobles removed in order.
- If no such node exists, print -1.

Sample Input:

```
6
-1 0
1 1
1 1
3 1
3 0
3 0
```

Sample Output:

```
2 4
```

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> adj[n + 1], res(n + 1);
    for(int i = 1; i <= n; i++) {
        int p, r; cin >> p >> r;
        res[i] = r;
        if(p != -1) adj[p].push_back(i); }
    bool found = false;
    for(int i = 1; i <= n; i++) {
        if(res[i] == 1) {
            int cnt = 0;
            for(int j = 0; j < adj[i].size(); j++) {
                if(res[adj[i][j]] == 0) cnt++; }
            if(cnt == 0) {
                cout << i << " ";
                found = true; } } }
    if(!found) cout << -1;
    return 0; }
```

Parcel Count Maximization

Problem Statement:

Mike is a logistics manager at a parcel delivery company. There are n delivery stops, each initially assigned a number of parcels represented by an array p .

The company allows the vans to load additional parcels at each stop using a permutation of the delivery stops from 1 to n .

Mike may choose exactly one permutation $q = [q_1, q_2, \dots, q_n]$, and then update each stop's parcel count using:

$$p[i] := p[i] + q[i]$$

Mike's goal is to maximize the number of delivery stops that end up with the same number of parcels after this operation.

Constraints:

- $1 \leq n \leq 200000$
- $1 \leq p[i] \leq 10^9$
- q is a permutation of numbers from 1 to n

Input Format:

- First line contains integer n
- Second line contains n integers representing array p

Output Format:

Print a single integer representing the maximum number of stops having equal parcel count after applying the permutation.

Sample Input:

```
6
1 1 1 1 1 2
```

Sample Output:

```
2
```



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> arr;
    unordered_map<int,int> mp;
    for(int i = 0; i < n; i++) {
        int x; cin >> x;
        if(mp[x] == 0) arr.push_back(x);
        mp[x]++; }
    sort(arr.begin(), arr.end());
    int i = 0, j = 0, ans = 0;
    while(i < arr.size()) {
        while(j + 1 < arr.size() && arr[i] + n > arr[j + 1]) j++;
        ans = max(ans, j - i + 1);
        i++; }
    cout << ans << endl;
    return 0; }
```

Nearest Meeting Cell

Problem Statement:

You are given a maze with N cells. Each cell may have multiple entry points but not more than one exit (i.e. edges are unidirectional).

The cells are numbered from 0 to N-1.

You are given an array Edge[] where:

- Edge[i] contains the cell reachable from cell i
- Edge[i] = -1 means there is no exit from cell i

Given two cells C1 and C2, find the nearest meeting cell that can be reached from both cells.

If multiple such cells exist, return the one with the smallest index.

If no meeting cell exists, return -1.

Constraints:

- $1 \leq N \leq 10^5$

Input Format:

- First line: Integer N
- Second line: N integers representing Edge[]
- Third line: Two integers C1 and C2

Output Format:

Print a single integer representing the nearest meeting cell.

Sample Input:

```
23
4 4 1 4 13 8 8 8 0 8 14 9 15 11 -1 10 15 22 22 22 22 22 21
9 2
```

Sample Output:

```
4
```

```

#include <bits/stdc++.h>
using namespace std;

void bfs(int src, vector<int>& edge, vector<int>& dis) {
    queue<int> q;
    dis[src] = 0;
    q.push(src);
    while(!q.empty()) {
        int x = q.front(); q.pop();
        int y = edge[x];
        if(y != -1 && dis[y] == INT_MAX) {
            dis[y] = dis[x] + 1;
            q.push(y); } } }

int main() {
    int n; cin >> n;
    vector<int> edge(n);
    for(int i = 0; i < n; i++) cin >> edge[i];
    int c1, c2;
    cin >> c1 >> c2;
    vector<int> dis1(n, INT_MAX), dis2(n, INT_MAX);
    bfs(c1, edge, dis1);
    bfs(c2, edge, dis2);
    int mn = INT_MAX, ans = -1;
    for(int i = 0; i < n; i++) {
        if(dis1[i] != INT_MAX && dis2[i] != INT_MAX) {
            int mx = max(dis1[i], dis2[i]);
            if(mx < mn) {
                mn = mx;
                ans = i; } } }
    cout << ans << endl;
    return 0; }

```

Largest Sum Cycle

Problem Statement:

You are given a maze with N cells. Each cell may have multiple entry points but not more than one exit.

You are given an array Edge[] of N integers, where:

- Edge[i] contains the cell reachable from cell i in one step
- Edge[i] = -1 means the ith cell has no exit

Your task is to find the sum of the largest cycle in the maze.

The sum of a cycle is defined as the sum of node indices present in that cycle.

Note:

- Cells are numbered from 0 to N-1

Constraints:

- $1 \leq N \leq 10^5$

Input Format:

- First line contains integer N
- Second line contains N integers representing Edge[]

Output Format:

Print a single integer representing the sum of nodes forming the largest cycle.

If no cycle exists, print -1.

Sample Input:

23

4 4 1 4 13 8 8 8 0 8 14 9 15 11 -1 10 15 22 22 22 22 22 21

Sample Output:

45

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> edge(n), indegree(n), vis(n, 0);
    for(int i = 0; i < n; i++) {
        cin >> edge[i];
        if(edge[i] != -1) indegree[edge[i]]++; }
    queue<int> q;
    for(int i = 0; i < n; i++) {
        if(indegree[i] == 0) q.push(i); }
    while(!q.empty()) {
        int x = q.front(); q.pop();
        if(edge[x] != -1) {
            int y = edge[x];
            indegree[y]--;
            if(indegree[y] == 0) q.push(y); } }
    int ans = -1;
    for(int i = 0; i < n; i++) {
        if(indegree[i] == 0 || vis[i]) continue;
        int j = i, sum = 0;
        while(!vis[j]) {
            sum += j;
            vis[j] = 1;
            j = edge[j]; }
        ans = max(ans, sum); }
    cout << ans << endl;
    return 0; }

```

Data Server Storage Consolidation

Problem Statement:

In a data center, servers are arranged in a rack with n server slots. However, due to varying server installations, the servers are spread out across different slots.

You want to consolidate the servers into consecutive slots to improve data access efficiency.

Find the minimum number of moves required to place all the servers into consecutive slots.

In one move, you can move one server to any arbitrary position.

Function Description:

Complete the solve function.

Parameters:

- $n \rightarrow$ number of servers
- $a[] \rightarrow$ array containing unique integers representing current server positions

Return:

Return an integer representing the minimum number of moves required to make all server positions consecutive.

Constraints:

- $1 \leq n \leq 10^5$
- $1 \leq a[i] \leq 10^9$

Input Format:

- First line contains integer n
- Next n lines contain integers representing server positions

Sample Input 1:

```
5
1
2
3
5
6
```

Sample Output 1:

```
1
```

Explanation:

Move server from position 6 to 4.

Sample Input 2:

4
3
5
7
9

Sample Output 2:

2

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> p(n);
    for(int i = 0; i < n; i++) cin >> p[i];
    sort(p.begin(), p.end());
    int j = 0, ans = n;
    for(int i = 0; i < n; i++) {
        while(j < n && p[j] - p[i] <= n - 1) j++;
        ans = min(ans, n - (j - i)); }
    cout << ans << endl;
    return 0; }
```

Power Distribution Network in Electrovia

Problem Description

In the growing city of Electrovia, the power distribution network is designed as a balanced tree, with a central power plant serving as the root. The cities are connected through power lines, and each power line has an energy transmission cost that depends on its maintenance and power loss over distance.

Objective

As the city's energy manager, you need to:

1. Monitor the transmission costs between different cities
2. Adjust them as necessary due to maintenance or upgrades

Functionality Requirements

1. Transmission Cost Query:
Calculate the total transmission cost between any two cities.
2. Cost Update Query:
Modify the transmission cost for any specific power line connecting two cities.

Input Format

Network Configuration

- First line contains 3 integers:
 - N (number of cities)
 - R (root city representing the power plant)
 - M (which equals N-1)
- The next N-1 lines each contain three integers U, V, W, where:
 - U and V are two connected cities
 - W is the transmission cost along the power line

Queries

- An integer Q indicating the number of queries
- Each of the next Q lines describes a query, either:
 - 1 A B 0 to calculate the transmission cost between cities A and B
 - 2 U V W to update the transmission cost between cities U and V to W

Output Format

Return the sum of results of all transmission cost queries (type 1 queries) between the specified cities


```

#include<bits/stdc++.h>
using namespace std;

vector<pair<int,int>> a[100005];

bool dfs(int u,int p,int d,int s,int &ans){
    if(u==d) return ans=s,1;
    for(auto &i:a[u])
        if(i.first!=p && dfs(i.first,u,d,s+i.second,ans))
            return 1;
    return 0;
}

int main(){
    int n,r,m,u,v,w,q,t,ans,sum=0;
    cin>>n>>r>>m;
    while(m--){
        cin>>u>>v>>w;
        a[u].push_back({v,w});
        a[v].push_back({u,w});
    }
    cin>>q;
    while(q--){
        cin>>t>>u>>v>>w;
        if(t==1) dfs(u,-1,v,0,ans),sum+=ans;
        else{
            for(auto &i:a[u]) if(i.first==v) i.second=w;
            for(auto &i:a[v]) if(i.first==u) i.second=w;
        }
    }
    cout<<sum;
}

```

Building Distance Analysis

Problem Statement:

In a large city, buildings are arranged in a grid with n rows and m columns.

Each building in this grid has a specific `building_type` represented by an integer from 1 to 100000.

City planners want to analyze the distance between all buildings of the same type to better understand the distribution of building clusters.

The Manhattan distance between two buildings located at $(r1, c1)$ and $(r2, c2)$ is the total number of horizontal and vertical steps required to reach one from the other.

The task is to calculate the sum of the Manhattan distances between each pair of buildings of the same type for every unique building type in the grid.

Input:

- First line contains two integers n and m
- The next n lines each contain m integers representing the building types

Output:

Print a single integer representing the sum of Manhattan distances between each pair of buildings of the same type.

Constraints:

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 10^5$
- $1 \leq a[i][j] \leq 10^5$
- $n \times m \leq 10^5$

Sample Input:

```
2
3
1 2 3
3 2 1
```

Sample Output:

```
7
```

Sample Input:

```
3
4
1 1 2 2
2 1 1 2
2 2 1 1
```

Sample Output:

76

```
#include <bits/stdc++.h>
using namespace std;
vector<int> x[100005], y[100005];

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> grid(n, vector<int>(m));
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            cin >> grid[i][j];
            int t = grid[i][j];
            x[t].push_back(i);
            y[t].push_back(j); } }
    long long ans = 0;
    for(int t = 1; t <= 100000; t++) {
        if(x[t].size() == 0) continue;
        sort(x[t].begin(), x[t].end());
        sort(y[t].begin(), y[t].end());
        vector<long long> sufX(x[t].size()), sufY(y[t].size());
        for(int j = x[t].size() - 1; j >= 0; j--) {
            sufX[j] = x[t][j];
            sufY[j] = y[t][j];
            if(j != x[t].size() - 1) {
                sufX[j] += sufX[j + 1];
                sufY[j] += sufY[j + 1]; } }
        for(int j = 0; j + 1 < x[t].size(); j++) {
            ans += sufX[j + 1] - 1LL * (x[t].size() - 1 - j) * x[t][j];
            ans += sufY[j + 1] - 1LL * (y[t].size() - 1 - j) * y[t][j]; } }
    cout << ans << endl;
    return 0; }
```

Network Technician's Mission

Problem Description

In the city of Techville, a network technician named Sam is tasked with inspecting a newly established network of communication towers.

The network is structured as a rooted tree with n towers, with the main control tower located at node 1. This control tower serves as the central hub for all communications.

The network is built through various terrains, some of which are marked as hazardous zones.

Each tower is either in a safe zone (1) or a hazardous zone (0).

The outermost towers (the leaf nodes of the tree) need to be inspected by Sam, but he is cautious and wants to avoid paths that traverse through too many consecutive hazardous zones.

Your task is to help Sam count the number of outermost towers (leaf nodes) he can safely inspect, where the path from the main control tower to the leaf contains no more than M consecutive hazardous zones.

It is guaranteed that the given set of edges forms a tree structure.

Function Description

Parameters:

- $N \rightarrow$ number of nodes in the tree
- $M \rightarrow$ maximum allowed consecutive hazardous zones
- $K \rightarrow$ always equal to $N-1$
- $Arr1 \rightarrow$ array of size N containing 0 or 1 for each node
- $Arr2 \rightarrow$ edges of the tree

Constraints:

- $2 \leq N \leq 10^5$
- $1 \leq M \leq N$
- $K = N-1$
- Each element in $Arr1$ is either 0 or 1

Input Format:

- First line contains N
- Second line contains M
- Third line contains K
- Next N integers describe $Arr1$
- Next K lines contain edges

Output Format:

Print the number of valid leaf nodes Sam can inspect.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void dfs(int u,int p,int c,vector<vector<int>>&a,vector<int>&v,int&m,int&ans){
    c=v[u]?0:c+1;
    if(c>m)return;
    bool f=0;
    for(int x:a[u])if(x!=p)f=1,dfs(x,u,c,a,v,m,ans);
    if(!f)ans++;
}
```

```
int main(){
    int n,m,k,ans=0;
    cin>>n>>m>>k;
    vector<int>v(n+1);
    for(int i=1;i<=n;i++)cin>>v[i];
    vector<vector<int>>a(n+1);
    while(k--){
        int u,v;
        cin>>u>>v;
        a[u].push_back(v);
        a[v].push_back(u);
    }
    dfs(1,-1,0,a,v,m,ans);
    cout<<ans;
}
```

Smart City Traffic Toll System

Background

In a smart city, there are an infinite number of traffic control hubs numbered with positive integers starting from 1.

Each hub is connected by smart roads in a unique tree-like structure:

1. There is a direct, bidirectional road between hub i and $2i$.
2. Another direct, bidirectional road exists between hub i and $2i + 1$.

Given this structure, there is always a unique shortest path between any two traffic control hubs.

Initially, passing through any road is toll-free.

However, to optimize traffic flow and manage congestion, the city traffic authority occasionally imposes toll fees on certain roads along specific paths.

The traffic authority will introduce a series of changes:

1. Toll Fee Update:
An update described by integers x , y , and t imposes a toll of t units on all roads along the shortest path from hub x to hub y .
2. Travel Cost Calculation:
A commuter travels from hub x to hub y using the shortest path, and you need to calculate the total toll fees they incur.

Input

- First line contains an integer q ($1 \leq q \leq 100000$), representing the number of events.
- Next q lines each contain 4 integers:
 - If first integer is 1:
 $1 \ x \ y \ t$
impose toll t on all roads from hub x to hub y .
 - If first integer is 2:
 $2 \ x \ y \ \theta$
calculate total cost from hub x to hub y .

Output

For each travel query ($2 \ x \ y \ \theta$), compute the trip cost and add it to a running sum.

Print the final accumulated sum.

Sample Input:

3

1 1 2 4
1 2 4 2
2 1 4 0

Sample Output:

6

```
#include<bits/stdc++.h>
using namespace std;
```

```
int main(){
    int q,t; long long x,y,w,ans=0; cin>>q;
    map<pair<long long,long long>,long long> c;
    while(q--){
        cin>>t>>x>>y>>w;
        vector<long long>a,b;
        for(;x/=2)a.push_back(x);
        for(;y/=2)b.push_back(y);
        reverse(a.begin(),a.end()); reverse(b.begin(),b.end());
        int i=0;
        while(i<a.size()&&i<b.size()&&a[i]==b[i])i++;
        if(t==1){
            for(int j=-i;j<a.size()-1;j++)c[{a[j],a[j+1]}]+=w,c[{a[j+1],a[j]}]+=w;
            for(int j=i;j<b.size()-1;j++)c[{b[j],b[j+1]}]+=w,c[{b[j+1],b[j]}]+=w;
        }else{
            long long s=0;
            for(int j=-i;j<a.size()-1;j++)s+=c[{a[j],a[j+1]}];
            for(int j=i;j<b.size()-1;j++)s+=c[{b[j],b[j+1]}];
            ans+=s;
        }
    }
    cout<<ans;
}
```